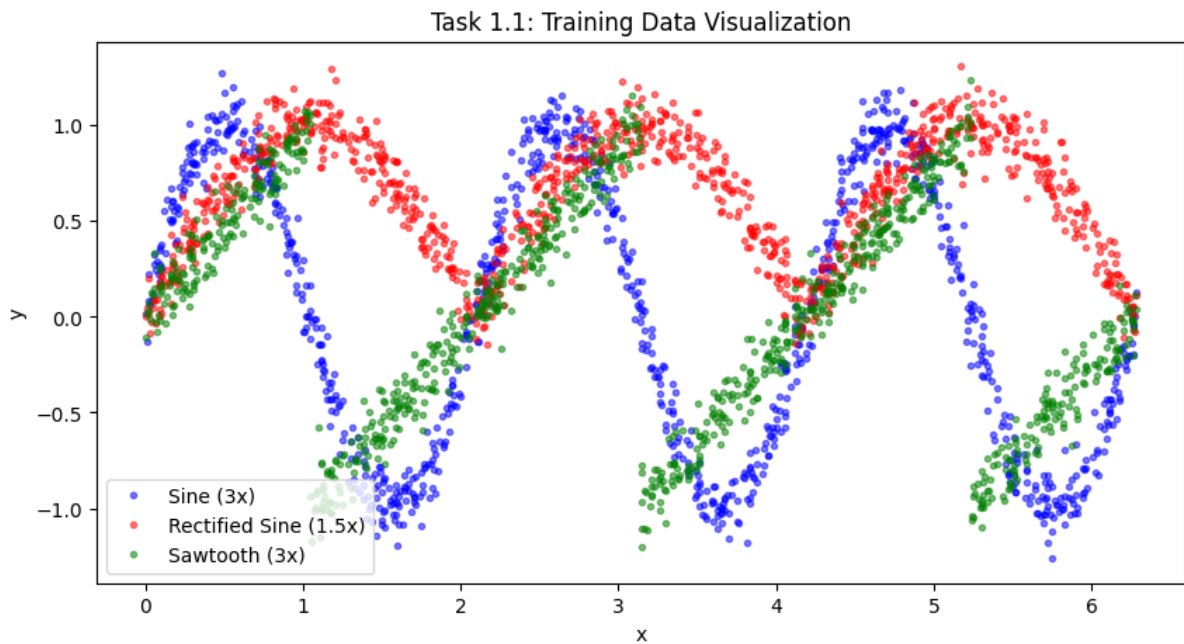


Module 3 Post-Lab Report

학번 비공개 심현성

[Lab 1]

[Task 1.1]



파란색: Sine function, 빨간색: Rectified Sine function, 초록색: Sawtooth function

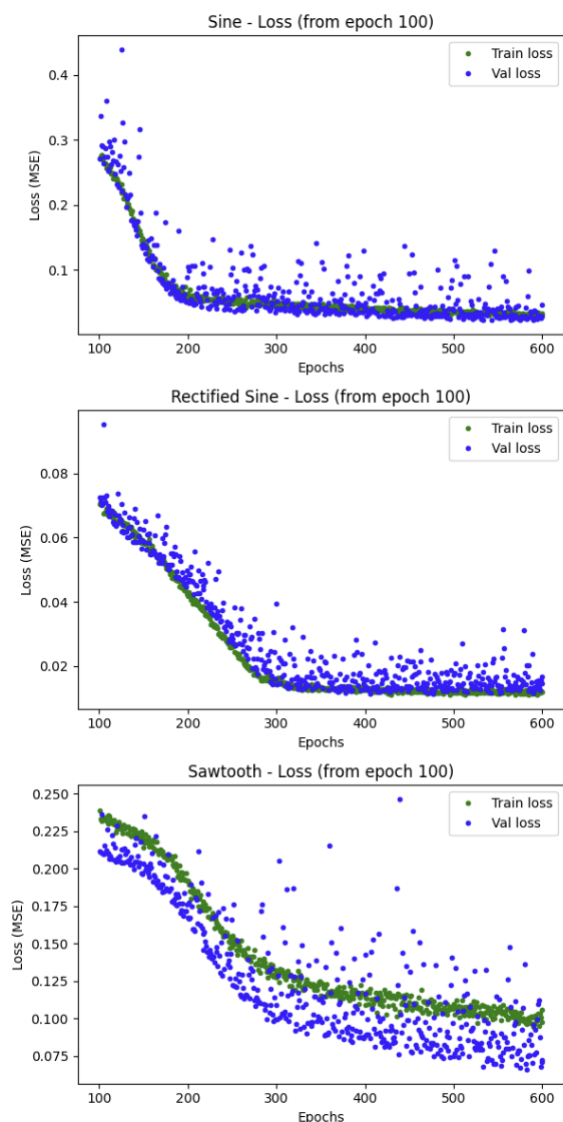
[Task 1.2]

모든 함수에 아래의 동일한 모델 구조를 사용하였다. 입력층에서는 입력값을 받아들이며, 두 개의 hidden layer는 각각 64개의 neuron과 ReLU 활성화 함수를 사용한다. ReLU는 비선형성을 부여하여 신경망이 단순한 직선 관계가 아닌 복잡한 함수 형태를 학습할 수 있도록 한다

```
1 def build_model():
2     model = tf.keras.Sequential()
3     model.add(layers.Dense(64, activation='relu', input_shape=(1,)))
4     model.add(layers.Dense(64, activation='relu'))
5     model.add(layers.Dense(1))
6     model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
7     return model
8
9 model_sine = build_model()
10 model_rect = build_model()
11 model_saw = build_model()
12
13 model_sine.summary()
14
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 64)	128
dense_3 (Dense)	(None, 64)	4,160
dense_4 (Dense)	(None, 1)	65

Optimizer: RMSProp, Loss: MSE, Metric: MAE, Batch size: 16, Epoch: 600



좌측 figure는 각 dataset에 대한 training & validation loss를 나타낸 그래프이다. 세 함수 중 sawtooth function이 가장 학습하기 어려웠다. 동일한 epoch(600) 동안 학습시켰을 때 loss가 flat에 도달한 sine과 rectified sine과는 달리, sawtooth는 여전히 training loss가 감소 중이었다. 또한, validation loss가 training loss 근처 혹은 살짝 그 윗부분에 몰려 있는 sine이나 rectified sine과는 달리, validation loss의 분산이 상대적으로 크고 epoch에 따른 fluctuation 역시 크게 나타났다. training loss가 평균적으로 validation loss보다 아래에 있지 않아 상대적으로 불안정한 학습 양상을 보였다. 이러한 결과의 원인은 sawtooth 함수의 discontinuity 때문이다. Sine은 연속적이고 smooth한 곡선이라 학습이 매우 안정적이며, rectified sine은 연속적이지만 일부 점에서 smooth하지 않은 곡선이라 flat에 도달하는 시점이 더 길기는 하나 신경망이 패턴을 학습하는 데에 문제는 없다. 반면 sawtooth는 주기마다 급격한 불연속 점프가 존재하여, 연속함수의 조합으로 이루어진 신경망으로 이를 표현하는 데는 본질적으로 어려움이 있다. 따라서 함수의 discontinuity 여부가 모델 성능에 큰 영향을 미친다고 볼 수 있다.

[Task 2.1]

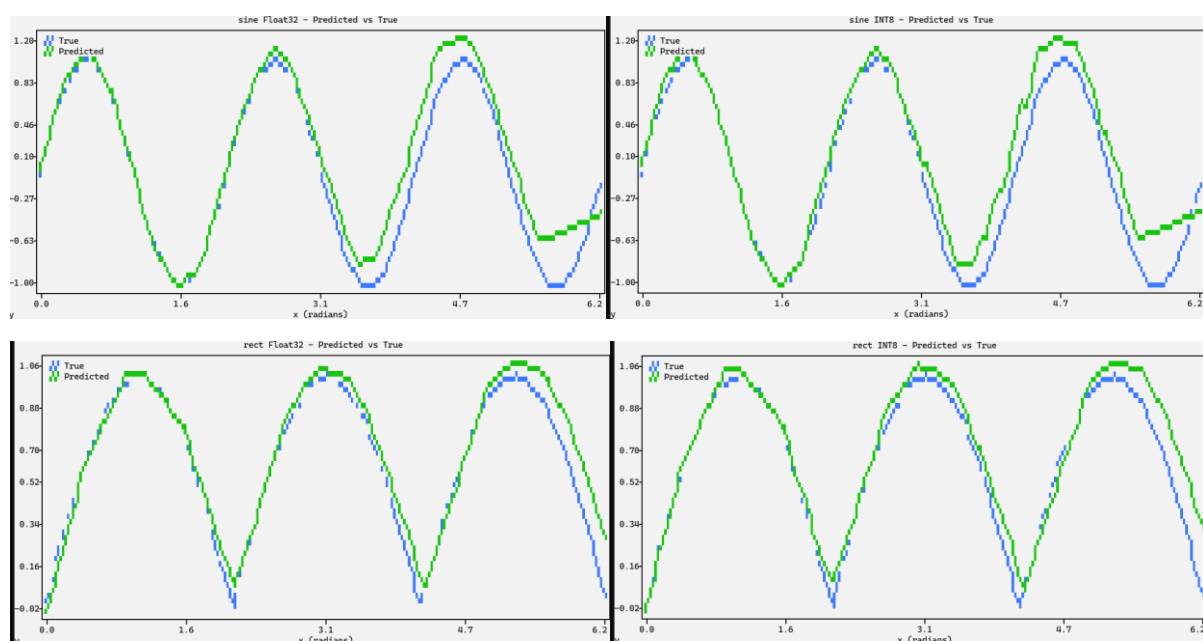
Model	Loss (MSE)	MAE
Sine Float32	0.050471	0.170573
Sine INT8	0.052703	0.178198
Rect Float32	0.017338	0.102989
Rect INT8	0.018673	0.105610
Sawtooth Float32	0.075768	0.178208
Sawtooth INT8	0.078323	0.184039

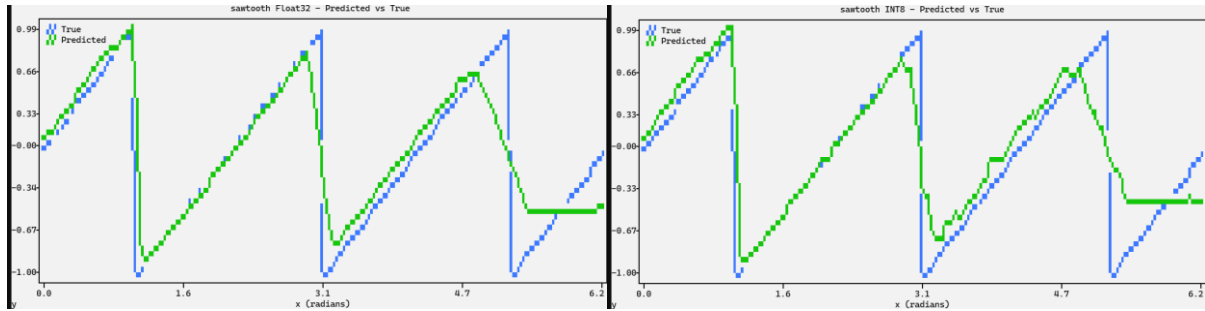
6개 모델의 test set에 대한 loss와 MAE는 좌측 figure와 같다. 대체적으로 같은 dataset에 대해서는 loss와 MAE가 크게 달라지지는 않고 유사하나, 양자화된(int8) 버전의 loss와 MAE가 약간 더 큰 경향성으로 보인다. 학습 시 validation set에 대한 성

능과 위의 test set에 대한 성능을 비교하자면, sine 함수의 최종 epoch(600)에 대한 validation loss, MAE 값은 각각 0.0464, 0.1631이고, rectified sine 함수의 최종 validation loss, MAE 값은 각각 0.0166, 0.1026이며, sawtooth 함수의 최종 validation loss, MAE 값은 각각 0.0711, 0.1743이다. 이 validation set에 대한 성능은 test set에 대한 성능과 유사함을 확인할 수 있다.

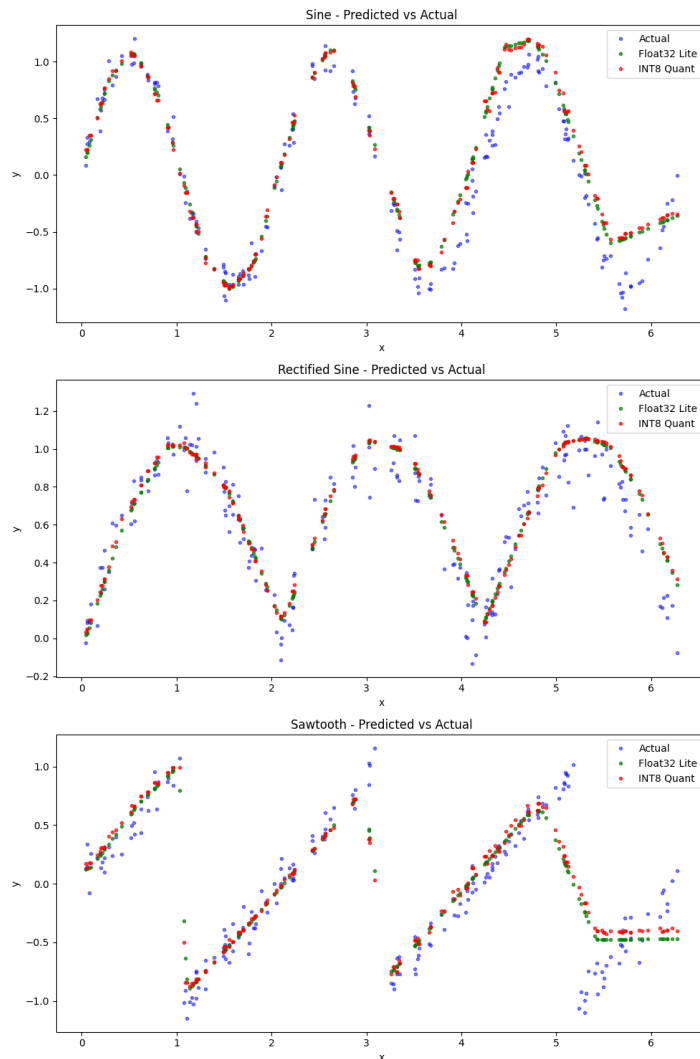
[Task 3.1]

Model Dataset	Model Type	File Size (bytes)	Mean (ms)	SD (ms)	Median (ms)	RMSE	MAE	Max Error
Sine	Float32	19324	0.0353	0.0059	0.0347	0.2016	0.1515	0.4624
Sine	INT8	9992	0.0465	0.0003	0.0465	0.2073	0.1611	0.4850
Rectified	Float32	19324	0.0347	0.0004	0.0347	0.0791	0.0563	0.2610
Rectified	INT8	9992	0.0474	0.0060	0.0467	0.0888	0.0631	0.2954
Sawtooth	Float32	19324	0.0354	0.0040	0.0351	0.2829	0.1647	1.3974
Sawtooth	INT8	9992	0.0465	0.0003	0.0464	0.2845	0.1707	1.3699





위 6개 figures는 좌측상단부터 우측으로 각각 sine float32, sine int8, rectified float32, rectified int8, sawtooth float32, sawtooth int8에 대한 true values와 test set의 predicted outputs 결과이다.



좌측 figure는 training data와 Colab에서의 float32, int8 모델의 predicted outputs 결과이다. 위의 Pi에서의 결과와 비교해보면 거의 일치하며 본 함수의 전체적인 형태를 잘 따라감을 확인할 수 있다. 또한, 성능 역시 [Task 2.1]의 table(Colab에서의 test set에 대한 성능)과 [Task 3.1]의 table(Pi에서의 test set에 대한 성능)을 비교해보면 MSE와 RMSE², MAE 값이 유사함을 확인할 수 있다. 성능 지표 수치만 봤을 때는 Pi benchmark의 error가 약간 더 낮게 나타났으나 이는 Colab에서 test set이 true sine 함수(혹은 다른 함수)가 아니라 noise가 10% 추가된 값이기 때문에 error metric이 상대적으로 크게 나타난 것으로 보인다. Colab에서와 Pi에서의 predicted waveform들을 비교해보면 Colab에서의 그래프가 약간 더 smooth하고 안정되어 보인다.

Colab과 Pi 모두 양자화(int8)시키면 성능 지표(MSE, RMSE, MAE)가 미세하게 저하되기는 하나 체감될 수준은 아닌

것으로 보인다. Pi에서 추론 속도를 비교해보면 양자화 시 file size는 절반 가량 감소하고 추론 속도는 약간 저하(소요 시간 증가)하였다. 추론 속도의 경우 이론적으로는 양자화 시의 속도가 더 빠를 것으로 예상하였으나 실제 결과는 달랐는데 이는 모델이 너무 작아 연산량 자체가 거의 없고 interpreter overhead 혹은 quant/dequant overhead가 더 크게 작용한 것으로 보인다.

[Task 3.2]

Deployment scenario 1: 배터리 기반 embedded device에서는 model size와 power efficiency가 매우 중요하다. 따라서 INT8 quantized model이 더 적합하다. 본 실험에서 양자화 이후 모델 크기는 절반 가량으로 감소하였다. Embedded system에서는 storage와 memory resource가 제한적이므로 이러한 크기 감소는 매우 중요한 장점이다. Inference speed 측면에서는 본 실험에서 INT8이 Float32보다 오히려 약간 느렸다(Sine 기준 0.0465ms vs 0.0353ms). 이는 모델이 너무 작아 quant/dequant overhead가 연산량 절감 효과보다 크게 작용했기 때문이다. 그러나 모델 규모가 커질수록 INT8 연산의 이점이 커질 가능성이 있으며, 현재 실험에서의 속도 차이도 절대적으로는 무시할 수 있는 수준(0.01ms 차이)이다. Accuracy 측면에서는 양자화 이후 RMSE와 MAE가 증가하였으나 증가 폭이 크지 않았으며 전체적인 waveform prediction은 여전히 안정적으로 유지되었다. 따라서 양자화 버전이 더 deployment에 적합한 선택이다.

Deployment scenario 2: Safety-critical system에서는 storage나 power efficiency보다 prediction accuracy와 reliability가 더 중요하다. 따라서 이 경우 Float32 model이 더 적합하다. 본 실험 결과 모든 함수에 대해 Float32 model이 Int8 model보다 더 낮은 MSE와 RMSE, MAE를 기록하였다. Inference speed 측면에서는 일반적으로 INT8 연산이 더 효율적일 가능성이 있으나, 콘센트 전원 환경에서는 추론 속도로 인한 전력 소비 제약이 없으므로 이는 선택 기준에서 중요하지 않다. Model size 측면에서도 Float32가 INT8보다 크지만, 마찬가지로 콘센트 전원 환경에서는 storage 제약이 상대적으로 덜 중요하므로 허용 가능하다. 따라서 accuracy가 가장 중요한 선택 기준이 되며, 더 낮은 MSE, RMSE, MAE를 기록한 Float32 model이 적합한 선택이다.

[Lab 2]

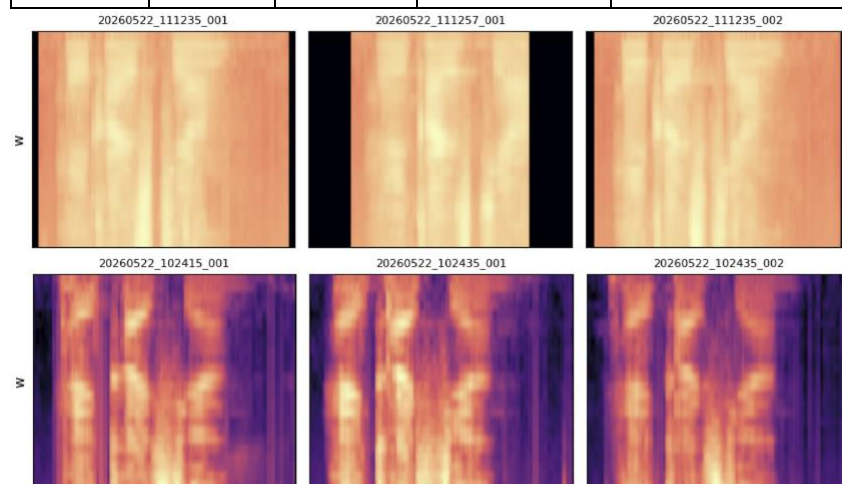
[Task 1.1]

1. RMS_THRESHOLD_DB는 speech와 silence를 구분하는 threshold 역할을 한다. 이 값이 낮을수록 작은 소리도 speech로 인식하고, 값이 높을수록 큰 소리만 speech로 인식한다. 시끄러운 실험실 환경에서는 배경 잡음이 speech로 잘못 검출되는 것을 방지하기 위해 RMS_THRESHOLD_DB를 증가시켜야 한다. 그러나 threshold를 너무 높게 설정하면 작은 목소리나 멀리서 말한 wake word가 검출되지 않을 수 있다. 반대로 threshold가 너무 낮으면 키보드 소리, 주변 대화 등의 잡음이 speech로 잘못 분리될 수 있다. 이러한 문제는 잘못된 training sample 생성과 모델 성능 저하로 이어질 수 있다.

2. Recording이 시작되는 순간과 종료되는 순간에는 segment들이 잘릴 가능성이 높다. Recording이 시작되기 전에 사용자가 이미 말하고 있을 수 있으며, recording 종료 직전에 speech가 끝까지 저장되지 못할 수 있다. 이러한 truncated sample은 wake word 일부 phoneme이 누락된 불완전한 데이터가 되므로 모델 학습 품질을 저하시킬 수 있다. 따라서 첫 번째와 마지막 segment를 제거함으로써 dataset 품질을 향상시킬 수 있다.

[Task 1.2]

Category	Final samples	Removed samples	Common removal reasons	Variation strategies used	Issues/problems encountered
Wake word	100	11	Truncated speech, low volume	Speed variation, distance variation, emphasis variation, pitch variation	Segmentation failure, merged utterances
Other words	100	6	Incorrect segmentation	Greetings, sound-alike phrases, random phrases	Noise falsely detected as speech
Noise	102	2	Accidental speech included	Keyboard typing, footsteps, desk scratching/tapping, ambient noise	Inconsistent background noise level



좌측 figure는 나(상단)와 partner(하단)의 wake word 데이터의 spectrogram들이다.

세 쌍 모두 공통적으로 상단 spectrogram이 상대적으로 더 밝게 나타났으며, 이는 더 강한 acoustic energy 및 louder speech를 시사한다.

세 쌍 중 왼쪽과 오른쪽 spectrogram pairs는 temporal rhythm과 acoustic structure가 유사하게 나타났다. 특히 어둡거나 밝은 세로 energy band가 비슷한 시간 축 상에 나타났으며, 이는 syllable timing과 speaking pattern이 유사함을 의미한다. Pronunciation과 speaking speed 역시 비슷한 경향을 보였다.

반면 가운데 pair는 양옆 pairs와 비교했을 때 temporal structure에서 차이를 보였다. 특히 어둡거나 밝은 세로 energy band의 위치와 간격이 다르게 나타났으며, 상단 spectrogram은 양옆에 상대적으로 큰 silence region이 존재하였다. 이는 상단에서의 wake word가 더 짧고 빠르게 발화되었음을 의미한다. 또한 speaking rhythm, pronunciation, emphasis 차이 역시 관찰되었다.

Single-speaker 데이터만으로 학습된 모델은 wake word 자체의 일반적인 acoustic pattern보다 특정 화자의 pitch, speaking style, pronunciation 특성을 학습할 가능성이 있다. 따라서 새로운 화자에 대해 성능이 크게 감소할 수 있으며, 이는 generalization 부족 문제를 의미한다.

[Task 2.1]

Model	Training samples	Test samples	Test accuracy	Epochs to converge
Baseline model	240	60	81.7%	about 15 epochs
Augmented model	1200	300	94.7%	about 25 epochs

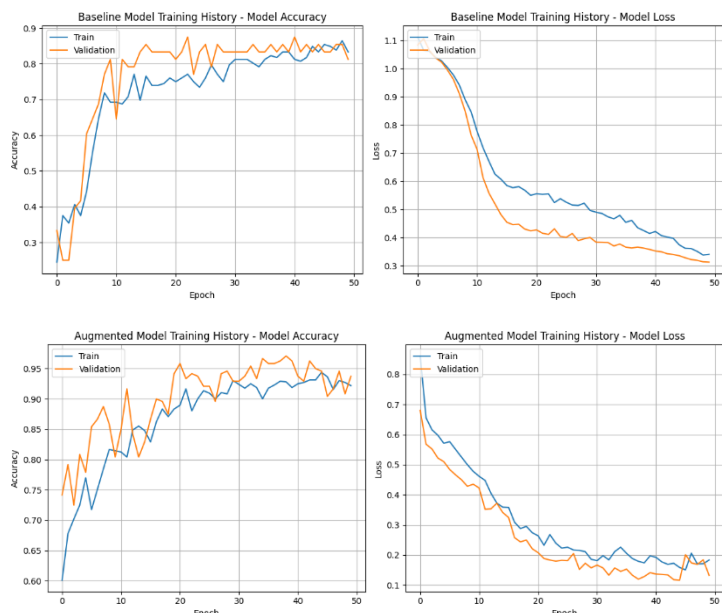
1. Baseline model은 category별 원본 sample만 사용하여 학습되었다. 각 category(wake word, other words, noise)에서 약 100개씩 총 약 300개의 sample을 활용하였으며, train/test split 비율이 8:2이므로 약 240개의 sample이 training set으로 분리되었다. 이후 model.fit()에서 validation_split=0.2가 적용되었기 때문에 이 240개 중 약 192개가 실제 weight update에 사용되었고 약 48개가 validation에 사용되었다.

Augmented model은 augmentation factor = 5를 적용하여 원본 sample 300개가 약 1500개로 증가한 dataset을 사용하였다. train/test split 이후 약 1200개의 sample이 training set으로 분리되었으며, validation_split=0.2 적용 후 약 960개가 실제 weight update에 사용되고 약 240개가 validation에 사용되었다.

따라서 위 table에서는 "training set samples" 기준으로 baseline은 240개, augmented는 1200개로 정리할 수 있다. 다만 실제 epoch별 gradient update에 사용된 sample 수는 validation split을 제외한 baseline 192개, augmented 960개이며, 이는 training log에서 baseline은 epoch당 6 steps, augmented model은 epoch당 30 steps로 나타난 것과도 일치한다.

Baseline model의 최종 test accuracy는 81.7%였으며, augmented model의 최종 test accuracy는 94.7%였다.

Convergence epoch는 단순히 best checkpoint epoch가 아니라, 우측 두 figure(상단: baseline, 하단: augmented)를 참고하여 accuracy와 loss의 improvement가 크게 감소하여 training curve가 plateau 형태를 보이기 시작한 시점을 기준으로 판단하였다. Baseline model의 경우 약 15 epoch 이후 accuracy와 loss의 improvement가 완만해졌으며, augmented model의 경우 25 epoch 이후 accuracy와 loss가 안정화되는 경향을 보였다.



2. Augmentation은 본 실험에서 매우 유용하였다. Augmented model은 baseline model에 비해 test accuracy가 13.0%p 더 높았다. 이는 augmentation이 limited dataset의 variation을 증가시켜 특정 data pattern에 대한 overfitting을 줄이고, 모델이 더 다양한 audio pattern에 대응하도록 도

왔기 때문이다. 제공된 training script는 noise addition, time shift, volume change, time stretch와 같은 augmentation을 사용하였다. 이러한 변형은 실제 사용 환경에서 발생할 수 있는 음량 차이, 말하는 속도 차이 등을 일부 모사한다. 따라서 augmentation은 단순히 data 수를 증가시키는 역할뿐만 아니라, wake word detector의 generalization 성능과 robustness를 향상시키는 데 중요한 역할을 한다.

Training logs를 보면 baseline model은 epoch당 6 steps, augmented model은 epoch당 30 steps를 수행하였다. 따라서 augmentation을 적용하면 한 epoch에서 처리하는 data 수가 5배 증가하므로 training time도 그만큼 증가한다. 학습 소요 시간 증가량이 크기는 하나 성능 13.0%p 향상은 무시할 수 없는 큰 개선 수치이기에 충분히 감당할 만한 tradeoff라고 판단한다.

제공된 script에 포함되지 않은 augmentation 중 robustness를 향상시킬 수 있는 방법으로는 pitch shift가 있다. 현재 script는 noise addition, time shift, volume change, time stretch 등을 통해 음량과 말하는 속도 variation은 반영하지만, 사람마다 다른 목소리 높낮이 차이는 직접적으로 반영하지 않는다. 기존 wake word wav data의 pitch를 약간 높이거나 낮추면 화자의 성별 차이, 개인별 voice tone 차이 등을 일부 모사할 수 있으며, 이를 통해 모델이 특정 화자의 음성 특성에 과하게 의존하는 문제를 줄일 수 있다. 따라서 pitch shift augmentation은 wake word detector의 cross-speaker generalization과 robustness 향상에 도움이 될 수 있다.

3. 아래 table은 네 모델의 file size, test accuracy, average inference time을 나타낸 표이다.

Model	File size	Test accuracy	Avg. Inference time (ms)
Baseline quantized	13.8 KB	80.0%	0.19 ms
Baseline float32	22.6 KB	81.7%	0.14 ms
Augmented quantized	13.8 KB	94.0%	0.09 ms
Augmented float32	22.6 KB	94.7%	0.07 ms

4. Quantization은 model size를 22.6 KB에서 13.8 KB로 크게 감소시켰다. 따라서 storage와 memory가 제한된 embedded system 환경에서는 quantization이 중요한 장점을 제공한다. 반면 accuracy는 baseline 기준 81.7%에서 80.0%로, augmented 기준 94.7%에서 94.0%로 소폭 감소하였다. 이는 quantization 과정에서 numerical precision이 감소하기 때문으로 볼 수 있다. Inference time 측면에서는 quantized model이 float32 model보다 반드시 더 빠르지는 않았다. 이는 모델 규모가 매우 작기 때문에 quantization/dequantization overhead의 영향이 상대적으로 크게 작용했기 때문으로 해석할 수 있다.

Data augmentation은 성능 향상에 매우 큰 영향을 주었다. Baseline model은 float32 기준 81.7%의 accuracy를 보였지만, augmented float32 model은 94.7%의 accuracy를 보였다. Quantized model에서도 baseline은 80.0%, augmented는 94.0%로 큰 성능 차이를 보였다. 이는 augmentation이 다양한 speaking speed, volume, temporal variation 등을 반영하여 모델의 generalization 성능과 robustness를 향상시켰기 때문으로 해석할 수 있다.

Colab 결과만 기준으로 하면 augmented float32 model을 deployment 후보로 선택할 수 있다. 이 모델은 네 모델 중 가장 높은 accuracy인 94.7%를 보였으며, average inference time 역시 0.07 ms

로 가장 빨랐다. Quantized model은 더 작은 file size를 제공하였지만, 현재 실험에서는 inference speed와 accuracy 측면에서 float32 model이 더 좋은 결과를 보였다. 또한 augmented float32 model의 file size 역시 22.6 KB로 매우 작은 수준이므로, Raspberry Pi 환경에서는 memory 부담이 크지 않을 것으로 예상된다. 따라서 현재 실험 결과만 고려하면 augmented float32 model이 가장 좋은 overall performance를 제공한다고 판단하였다.

[Task 3.1]

Model	Size (KB)	Accuracy (%)	FPR (%)	FNR (%)	Mean (ms)	SD (ms)	Median (ms)
Baseline float32	22.6	85.43	9.41	23.00	3.744	0.673	3.392
Baseline Quantized	13.8	85.76	9.90	21.00	5.169	0.403	5.149
Augmented float32	22.6	96.69	4.46	0.00	4.106	0.463	4.098
Augmented Quantized	13.8	96.36	4.95	0.00	5.151	0.121	5.154

Table 1: Own test data

Model	Size (KB)	Accuracy (%)	FPR (%)	FNR (%)	Mean (ms)	SD (ms)	Median (ms)
Baseline float32	22.6	60.66	9.41	96.12	3.866	0.616	4.085
Baseline Quantized	13.8	60.33	9.90	96.12	5.205	0.577	5.168
Augmented float32	22.6	78.36	4.46	54.37	3.903	0.979	4.062
Augmented Quantized	13.8	80.66	4.95	46.60	5.160	0.128	5.153

Table 2: Partner test data

1. 본 실험에서는 제공된 wakeWordSimpleEval.py 코드를 확장하여 warm-up inference 제외 timing 측정, confusion matrix, false positive/false negative rate 계산 기능을 추가하였다. 실제 timing 측정 이전에 10회의 warm-up inference를 수행하고 timing 통계에서는 제외하였다. 또한 mean, standard deviation(SD), median, min/max inference time을 모두 계산하도록 수정하였다. Wake-word system에서는 단순 accuracy만으로는 실제 성능을 충분히 평가할 수 없으므로 confusion matrix를 기반으로 false positive rate(FPR)와 false negative rate(FNR)를 추가적으로 계산하였다.

2. 모든 모델은 own test data에서는 높은 성능을 보였지만, partner test data에서는 성능이 크게 감소하였다. 특히 baseline 모델의 경우 partner voice에서 false negative rate가 약 96%까지 증가하였다. 이는 모델이 특정 화자의 acoustic feature에 강하게 overfit되었음을 의미한다.

반면 data augmentation을 적용한 모델은 partner voice에 대해서도 훨씬 높은 성능을 보였다. 예를 들어 augmented quantized model은 partner accuracy 80.66%, FNR 46.60%를 기록하였으며, baseline 모델 대비 매우 큰 성능 향상이 나타났다. 이는 augmentation이 다양한 volume, position, noise, speech tempo variation을 artificial하게 생성하여 모델 generalization을 향상시켰기 때문이라고 볼 수 있다.

Quantization은 model size를 크게 감소시켰다. Float32 모델은 22.6 KB였지만 quantized 모델은 13.8 KB로 감소하였다. 반면 accuracy 감소는 거의 발생하지 않았다. 이는 wake word detection model이 작은 CNN 구조와 MFCC 기반 feature를 사용하기 때문에 INT8 quantization에 비교적 robust했기 때문으로 보인다.

Inference time 측면에서는 float32 모델이 약간 더 빨랐으나, quantized augmented model은 네 모델 중 가장 작은 standard deviation(예: SD = 0.121 ms)을 보이며 가장 안정적인 timing 특성을 나타냈다. 반면 float32 model은 큰 timing variance(예: SD = 0.979 ms)를 보였다. 이는 quantized model이 평균 inference speed는 다소 느리더라도 보다 stable하고 predictable한 execution timing을 제공할 수 있음을 시사한다. Wake word system에서는 평균 속도뿐 아니라 timing consistency 역시 중요하다. Timing fluctuation이 크면 real-time interaction에서 latency variation이 발생할 수 있으며, 일부 상황에서는 wake word response delay로 이어질 수 있기 때문이다. 따라서 실제 deployment 환경에서는 평균 inference speed뿐 아니라 stable하고 predictable한 execution time 또한 매우 중요한 요소라고 볼 수 있다.

[Appendix]

A. TrainWakeWordSimple.ipynb 주요 수정 code snippets

A.1. Modified training function

```
14 def train_model(augment=True, model_name='wake_word_model'):  
15     """Train wake word model"""  
16     print("Loading data...")  
17     X, y = load_dataset(augment, aug_factor=5)  
18     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)  
19  
20     print(f"\nTrain: {len(X_train)}, Test: {len(X_test)}")  
21     print(f"Input shape: {X_train[0].shape}\n")  
22  
23     model = create_model(X_train[0].shape, len(CATEGORIES))  
24     model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])  
25     model.summary()  
26  
27     print(f"\nTraining model: {model_name} (Augmentation: {augment})...")  
28     history = model.fit(X_train, y_train, validation_split=0.2, epochs=50, batch_size=32,  
29         | callbacks=[early_stopping])  
30  
31     test_acc = model.evaluate(X_test, y_test, verbose=0)[1]  
32     print(f"\nTest accuracy for {model_name}: {test_acc*100:.1f}%")  
33  
34     model.save(os.path.join(OUTPUT_PATH, f'{model_name}.h5'))  
35     return model, X_test, y_test, history
```

Training curve 시각화를 위해 history를 반환하도록 바꿨고, baseline/augmented 모델을 구분 저장하기 위해 model_name을 추가하였다.

A.2. Training history visualization

```
1 def plot_training_history(history, title):
2     """Plot training & validation accuracy and loss"""
3     plt.figure(figsize=(12, 5))
4
5     # Plot training & validation accuracy values
6     plt.subplot(1, 2, 1)
7     plt.plot(history.history['accuracy'])
8     plt.plot(history.history['val_accuracy'])
9     plt.title(f'{title} - Model Accuracy')
10    plt.ylabel('Accuracy')
11    plt.xlabel('Epoch')
12    plt.legend(['Train', 'Validation'], loc='upper left')
13    plt.grid(True)
14
15    # Plot training & validation loss values
16    plt.subplot(1, 2, 2)
17    plt.plot(history.history['loss'])
18    plt.plot(history.history['val_loss'])
19    plt.title(f'{title} - Model Loss')
20    plt.ylabel('Loss')
21    plt.xlabel('Epoch')
22    plt.legend(['Train', 'Validation'], loc='upper left')
23    plt.grid(True)
24
25    plt.tight_layout()
26    plt.show()
```

Task 2.1에서 “epochs to converge를 visualization으로 support하라”고 했기 때문에 plot_training_history 함수를 추가하였다.

A.3. Model-specific TFLite conversion

```
1 # =====
2 # TFLITE CONVERSION
3 # =====
4 def convert_to_tflite(model, X_test, quantize=True, model_name='wake_word'):
5     """Convert to TFLite (optionally quantized)"""
6     converter = tf.lite.TFLiteConverter.from_keras_model(model)
7
8     if quantize:
9         def representative_dataset():
10             for i in range(min(100, len(X_test))):
11                 yield [X_test[i:i+1].astype(np.float32)]
12
13         converter.optimizations = [tf.lite.Optimize.DEFAULT]
14         converter.representative_dataset = representative_dataset
15         converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
16         converter.inference_input_type = tf.int8
17         converter.inference_output_type = tf.int8
18
19     tflite_model = converter.convert()
20     suffix = 'quantized' if quantize else 'float32'
21     output_path = os.path.join(OUTPUT_PATH, f'{model_name}_{suffix}.tflite')
22
23     with open(output_path, 'wb') as f:
24         f.write(tflite_model)
25
26     print(f"TFLite model saved: {output_path} ({len(tflite_model)/1024:.1f} KB)")
27     return output_path
```

네 모델 파일을 구분 저장 가능하도록 model_name을 활용하여 변경하였다.

A.4. Four-model training/conversion/testing pipeline

```
1 os.makedirs(OUTPUT_PATH, exist_ok=True)
2
3 # --- 1. Baseline Model (no augmentation) ---
4 print("\n===== Training Baseline Model =====")
5 baseline_model, X_test_baseline, y_test_baseline, history_baseline = train_model(augment=False, model_name='wake_word_baseline')
6 plot_training_history(history_baseline, "Baseline Model Training History")
7
8 # Convert Baseline Model
9 print("\n===== Converting Baseline Model (Float32) =====")
10 baseline_float_path = convert_to_tflite(baseline_model, X_test_baseline, quantize=False, model_name='wake_word_baseline')
11 print("\n===== Converting Baseline Model (Quantized) =====")
12 baseline_quantized_path = convert_to_tflite(baseline_model, X_test_baseline, quantize=True, model_name='wake_word_baseline')
13
14 # Test Baseline TFLite Models
15 print("\n===== Testing Baseline Quantized TFLite Model =====")
16 test_tflite(baseline_quantized_path, X_test_baseline, y_test_baseline)
17 print("\n===== Testing Baseline Float32 TFLite Model =====")
18 test_tflite(baseline_float_path, X_test_baseline, y_test_baseline)
19
20
21 # --- 2. Augmented Model (with augmentation) ---
22 print("\n===== Training Augmented Model =====")
23 augmented_model, X_test_augmented, y_test_augmented, history_augmented = train_model(augment=True, model_name='wake_word_augmented')
24 plot_training_history(history_augmented, "Augmented Model Training History")
25
26 # Convert Augmented Model
27 print("\n===== Converting Augmented Model (Float32) =====")
28 augmented_float_path = convert_to_tflite(augmented_model, X_test_augmented, quantize=False, model_name='wake_word_augmented')
29 print("\n===== Converting Augmented Model (Quantized) =====")
30 augmented_quantized_path = convert_to_tflite(augmented_model, X_test_augmented, quantize=True, model_name='wake_word_augmented')
31
32 # Test Augmented TFLite Models
33 print("\n===== Testing Augmented Quantized TFLite Model =====")
34 test_tflite(augmented_quantized_path, X_test_augmented, y_test_augmented)
35 print("\n===== Testing Augmented Float32 TFLite Model =====")
36 test_tflite(augmented_float_path, X_test_augmented, y_test_augmented)
```

네 모델 생성 및 저장 pipeline.

B. wakeWordSimpleEval.py 주요 수정 code snippets

B.1. Warm-up inference exclusion

```
DEFAULT_WARMUP_RUNS = 10 # Excluded from timing statistics

# ---- Warm-up (excluded from statistics) ----
print(f"\nRunning {num_warmup} warm-up inferences (excluded from timing)...")
x_warm = preprocess(test_samples[0]['signal'], input_details)
for _ in range(num_warmup):
    interpreter.set_tensor(input_details[0]['index'], x_warm)
    interpreter.invoke()
```

초기 model initialization 및 memory allocation에 따른 timing overhead를 제거하기 위해 warm-up inference를 추가하였다. Warm-up execution은 benchmark timing 통계에서 제외하였다.

B.2. Quantized preprocessing support

```
def preprocess(signal, input_details):
    """MFCC feature extraction + quantization handling."""
    signal = signal / (np.max(np.abs(signal)) + 1e-10)
    features = mfcc(signal, samplerate=SAMPLE_RATE, numcep=NUM_MFCC)
    features = features[..., np.newaxis]
    x = np.expand_dims(features, axis=0).astype(np.float32)

    input_scale, input_zero_point = input_details[0]['quantization']
    if input_scale > 0: # quantized model
        x = x / input_scale + input_zero_point
        x = x.astype(input_details[0]['dtype'])
    return x
```

MFCC feature extraction 및 INT8 quantization preprocessing을 별도의 preprocess() 함수로 분리하였다. 이를 통해 float32 및 quantized TFLite model 모두에 대해 일관된 preprocessing을 수행할 수 있도록 수정하였다.

B.3. Confusion matrix implementation

```
confusion = np.zeros((3, 3), dtype=int)

for sample in test_samples:
    x = preprocess(sample['signal'], input_details)

    start = time.perf_counter() # higher resolution than time.time()
    interpreter.set_tensor(input_details[0]['index'], x)
    interpreter.invoke()
    elapsed_ms = (time.perf_counter() - start) * 1000.0
    times.append(elapsed_ms)

    output = interpreter.get_tensor(output_details[0]['index'])
    pred = int(np.argmax(output))
    confusion[sample['label'], pred] += 1
```

Wake word, other word, noise 세 클래스에 대한 detailed classification analysis를 위해 confusion matrix를 추가하였다.

B.4. FPR/FNR calculation

```
# False Negative Rate: wake word missed (predicted as 0 or 2)
fn = int(confusion[WAKE_IDX, 1] + confusion[WAKE_IDX, 2])
fnr = fn / n_wake * 100 if n_wake > 0 else 0.0

# False Positive Rate: non-wake misclassified as wake
fp = int(confusion[1, WAKE_IDX] + confusion[2, WAKE_IDX])
fpr = fp / n_nonwake * 100 if n_nonwake > 0 else 0.0
```

Wake-word system에서는 단순 accuracy보다 false positive rate(FPR) 및 false negative rate(FNR)가 중요하므로, confusion matrix 기반 error metric 계산 기능을 추가하였다.

B.5. Extended timing statistics

```
# Inference time statistics
t = np.array(times)
mean_t = float(np.mean(t))
std_t = float(np.std(t, ddof=1)) if len(t) > 1 else 0.0
median_t = float(np.median(t))
min_t, max_t = float(np.min(t)), float(np.max(t))
```

Inference timing consistency 분석을 위해 mean latency뿐만 아니라 standard deviation, median, minimum, maximum latency를 추가적으로 계산하였다.

C. Modified scripts links

C.1. TrainWakeWordSimple.ipynb

https://drive.google.com/file/d/1lsFksj1JLBHrVU14W4qyfz2NTg6ASl1T/view?usp=drive_link

C.2. wakeWordSimpleEval.py

https://drive.google.com/file/d/1WgJCJ645wrMJwjuqgsOhCBuujEhshwJe/view?usp=drive_link

D. Data collection links

녹음 데이터 링크 비공개

[Lab 3]

[Task 2.1]

1. CONFIDENCE_THRESHOLD는 wake word로 판단하기 위해 필요한 최소 confidence 값이다. Threshold를 높이면 모델이 더 높은 확신을 가질 때만 detection을 수행하게 된다. 따라서 원래는 detection되던 애매한 신호들이 detection되지 않게 되어 false positive는 감소한다. 반면 실제 wake word를 말했더라도 confidence가 threshold를 넘지 못하는 경우가 증가하므로 false negative는 증가한다.

2. CONSECUTIVE_DETECTIONS는 confidence가 threshold를 초과한 상태가 몇 프레임 연속으로 유지되어야 최종 detection으로 인정할지를 결정한다. 즉 CONSECUTIVE_DETECTIONS는 연속된 high-confidence frame 수를 요구하여 일시적인 noise나 confidence spike를 제거하는 역할을 한다. 현재 시스템은 40 ms마다 inference를 수행하므로, 값을 15로 설정하면 약 600 ms 동안 threshold 초과 상태가 유지되어야 하므로 wake word조차 detection되기 어려워진다. 따라서 false positive는 감소하지만 false negative가 증가하며 detection latency도 증가한다.

3. COOLDOWN_FRAMES는 wake word가 한 번 검출된 이후 추가 detection을 무시하는 기간이다. Cooldown이 너무 길면 wake word가 검출된 이후 오랜 시간 동안 새로운 detection이 차단되므로, 실제 wake word를 다시 말해도 시스템이 반응하지 않을 수 있으며, 사용자는 시스템이 정상적으로 동작하지 않는다고 느낄 수 있다.

[Task 2.2]

streamWord_spring26.py에서 아래의 네 부분을 수정하였다.

```
# Logging setup
self.log_file = None
# Create a CSV log file when profile mode is enabled
if self.profile:
    fn = "log_" + datetime.datetime.now().strftime("%Y%m%d_%H%M%S") + ".csv"
    # Open log file in write mode
    self.log_file = open(fn, "w")
    # Write CSV header
    self.log_file.write("timestamp,confidence,peak_confidence\n")
```

StreamingSystem.__init__()에 log file 생성 기능을 추가하였다. Profile mode(--profile)가 활성화된 경우 timestamp를 포함한 CSV 파일을 생성하며, 파일의 첫 줄에는 header를 기록하도록 구현하였다.

```
# Record detection information to the log file
if self.log_file is not None:
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    # Store detection timestamp, current confidence, and peak confidence
    self.log_file.write(
        f"{timestamp},{confidence:.3f},{self.peak_confidence:.3f}\n"
    )
    # Immediately save data to disk
    self.log_file.flush()
```

Wake word가 검출될 때마다 detection 정보를 CSV 파일에 저장하도록 _on_detection()을 수정하였다. 저장되는 정보는 detection timestamp, current confidence, peak confidence이다.

```
def close_logging(self):
    """
    Close the log file if it is currently open.
    """
    if self.log_file is not None and not self.log_file.closed:
        self.log_file.close()

# Ensure the log file is properly closed before exiting
system.close_logging()
```

프로그램 종료 시 log file이 정상적으로 저장되도록 close_logging() 함수를 추가하였다. 또한 run_detection()의 finally block에서 해당 함수를 호출하여 파일이 안전하게 닫히도록 구현하였다.

Timestamp	Confidence	Peak confidence
20260522_153553	0.999	0.999
20260522_153555	0.995	0.999

20260522_153557	0.943	0.999
-----------------	-------	-------

Profile mode를 사용하여 프로그램을 실행한 후 wake word를 세 번 발화하였다. 생성된 CSV 파일에는 세 번의 detection 결과가 정상적으로 기록되었으며, 각 row에는 timestamp, confidence, peak confidence 정보가 포함되었다. 실험 결과, 모든 wake word detection이 정상적으로 기록되었으며 logging 기능이 의도한 대로 동작함을 확인하였다.

[Task 3.1]

1. Quiet 환경: 5분 동안 조용한 환경에서 실험한 결과 1회의 false positive가 발생하였다.

Timestamp	Confidence	Peak confidence	Hypothesis
20260522_152437	0.665	0.667	Brief background noise or distant speech

False positive rate는 5분 동안 1회 발생했으므로 시간당 약 12회로 추정된다.

2. Normal conversation 환경: 5분 동안 일반적인 대화 환경에서 실험한 결과 총 19회의 false positive가 발생하였다.

Timestamp	Confidence	Peak confidence	Hypothesis
20260522_152723	0.676	0.735	Short exclamation or interjection
20260522_152756	0.678	0.709	Rapid speech segment
20260522_152814	0.830	0.830	Similar-sounding phrase
20260522_152817	0.823	0.930	Similar-sounding phrase
20260522_152823	0.602	0.705	Loud background conversation
20260522_152904	0.758	0.758	Sudden change in speaking volume
20260522_152906	0.885	0.940	Similar-sounding phrase
20260522_152912	0.769	0.774	Similar-sounding phrase
20260522_152916	0.644	0.671	Sudden change in speaking volume
20260522_152929	0.632	0.632	Laughter mixed with speech
20260522_152953	0.697	0.697	Overlapping speech from multiple speakers
20260522_152958	0.745	0.745	Similar-sounding phrase
20260522_153015	0.734	0.734	Similar-sounding phrase
20260522_153034	0.665	0.682	Loud background conversation
20260522_153039	0.628	0.724	Short exclamation or interjection
20260522_153046	0.727	0.727	Similar-sounding phrase
20260522_153131	0.660	0.660	Sudden change in speaking volume
20260522_153200	0.681	0.681	Overlapping speech from multiple speakers
20260522_153218	0.804	0.804	Similar-sounding phrase

False positive rate는 5분 동안 19회 발생했으므로 시간당 약 228회로 추정된다.

3. 조용한 환경에서는 단 1회의 false positive만 발생하였으나, 일반 대화 환경에서는 19회의 false positive가 발생하였다. 이는 시스템이 환경 소음보다는 사람의 음성에 훨씬 민감하게 반응함을 의미한다. Wake word인 "Hey KAISTian"과 유사한 발음 또는 음운 구조가 일반 대화 속에 포함될 경우 모델이 이를 wake word로 잘못 인식하는 것으로 보인다. 반면 키보드 소리, 주변 기계음, 정적 배경 소음 등은 상대적으로 false positive를 거의 유발하지 않았다. 따라서 현재 시스템의 주요 false positive 원인은 환경 소음이 아니라 일반 음성 대화이며, 실제 사용 환경에서는 사람이 많이 대화하는 공간에서 오동작 가능성이 증가할 것으로 예상된다.

[Task 3.2]

Phrase	Detections	Avg. Confidence	Phonetic Similarity
"Hey question!"	3/3	0.764	High
"Hey Christian!"	3/3	0.692	High
"Hey Einstein!"	3/3	0.805	Medium
"A station!"	1/3	0.562	Medium
"Hello KAISTian!"	2/3	0.648	High

1. Why does the model confuse these phrases?: 본 실험에서 사용한 phrase들은 wake word인 "Hey KAISTian"과 일부 음운적 특징을 공유한다. 특히 "Hey question"과 "Hey Christian", "Hey Einstein"은 시작 부분의 "Hey"가 동일하며 끝부분의 "tion", "tian", "tein"의 발음이 "KAISTian"의 "Tian"과 같거나 비슷하기에 wake word와 유사한 리듬과 음향 패턴을 가진다. "Hello KAISTian" 역시 wake word와 동일한 단어를 포함하고 있기 때문에 높은 빈도로 false positive가 발생한다. 이는 모델이 정확한 단어 자체를 이해한다기보다, 입력 음성의 acoustic pattern과 phonetic similarity에 기반해 wake word 여부를 판단하기 때문으로 보인다.

2. What would you need to fix this?: 현재 실험에서 가장 현실적인 개선 방법은 parameter들을 조정하는 것이다. Threshold를 높이면 confidence가 낮은 false positive를 줄일 수 있으며, CONSECUTIVE_DETECTIONS 값을 증가시키면 순간적인 confidence spike에 의한 detection을 감소시킬 수 있다. 그러나 confidence가 매우 높은 phrase(예: "Hey question!", "Hey Einstein!")는 parameter 조정만으로 완전히 제거하기 어려울 수 있다. 따라서 parameter tuning은 false positive를 완화하는 데 도움을 주지만, 모든 adversarial phrase를 제거하는 것은 어렵다.

3. Would a higher threshold prevent these false positives? Would such a modification have any further impact on performance?: Threshold를 높이면 일부 false positive를 줄일 수 있다. 특히 confidence가 threshold 근처에서 발생하는 경우에는 잘못된 detection이 감소할 수 있다. 예를 들어 위 table을 참고하여 threshold를 0.8로 증가시키면, "Hey Einstein"을 제외한 다른 phrase들로 인한 false positive는 크게 감소한다. 그러나 "Hey Einstein"처럼 매우 높은 confidence를 생성하는 경우에는 threshold만으로는 완전히 해결하기 어렵다. Threshold를 지나치게 높이면 실제 wake word에 대한 detection도 감소하여 false negative가 증가하여 성능이 저하될 수도 있다.

4. What would be the real-world implications of this performance for a voice assistant? What is an acceptable false positive rate for a voice assistant?: 현재 시스템은 wake word와 발음이 유사한 phrase에 대해 비교적 높은 false positive rate를 보였다. 실제 voice assistant 환경에서는 일반 대화 중에도 의도하지 않은 활성화가 발생할 수 있으며, 이는 사용자 경험을 저하시킬 수 있다. 실제 상용 voice assistant는 상시 대기 상태로 동작하므로 사용자가 일상적인 대화를 하는 동안 거의 활성화되지 않을 정도의 매우 낮은 false positive rate가 요구된다. 개인적으로 하루에 1~2회 수준 이하의 false activation이 발생하는 경우가 acceptable하다고 생각한다. 반면 본 실험(task 3.1)의 conversation 환경에서는 약 228 detections/hour가 관측되었으므로 실제 상용 서비스 수준과는 큰 차이가 있다. 따라서 현재 시스템은 실험용으로는 충분히 의미 있는 성능을 보이지만, 상용 제품 수준으로 사용하기 위해서는 추가적인 parameter optimization이 필요하다.

[Task 3.3]

Distance	My voice	Partner's voice
Close (10 cm)	10/10	10/10
Normal (50 cm)	9/10	10/10
Far (1 m)	4/10	6/10

Volume	My voice	Partner's voice
Quiet speech	1/10	1/10
Normal speech	9/10	10/10
Loud speech	10/10	10/10

Environment	My voice	Partner's voice
Quiet room	9/10	10/10
Music playing	5/10	9/10
Lab noise	2/10	6/10

1. 실험 결과, 성능 저하는 주로 낮은 음량과 강한 배경 소음 환경에서 발생하였다. Quiet speech에서는 두 실험자 모두 1/10의 매우 낮은 detection rate를 보였으며, Lab noise 환경에서도 각각 2/10과 6/10에 그쳤다. 또한 거리 역시 성능에 영향을 주어, Close distance(10 cm)에서는 모두 10/10을 기록한 반면 Far distance(1 m)에서는 각각 4/10과 6/10으로 감소하였다.

특히 본 실험에서 가장 큰 영향을 준 변수는 volume으로 보인다. Quiet speech에서는 두 실험자 모두 1/10에 불과했지만 Loud speech에서는 모두 10/10을 기록하였다. 이는 wake word의 음향 특징이 충분한 에너지로 입력되지 않으면 MFCC 특징이 약해지고, 모델이 이를 안정적으로 인식하기 어려워지기 때문으로 판단된다.

반대로 시스템이 가장 강인한(immune) 특성을 보인 조건은 Close distance와 Loud speech였다. 두 조건 모두 거의 완벽한 detection rate를 기록하였으며, 화자에 따른 차이도 거의 나타나지 않았다. 따라서 현재 시스템은 입력 신호의 SNR(Signal-to-Noise Ratio)이 충분히 높은 환경에서는 안정적으로 동작하지만, 음량이 작거나 배경 소음이 큰 환경에는 매우 취약한 것으로 보인다.

2. 일부 조건에서는 두 실험자 간의 성능 차이가 뚜렷하게 나타났다. 예를 들어 Music Playing 환경에서는 각각 5/10과 9/10, Lab Noise 환경에서는 2/10과 6/10의 결과를 보였다. 반면 Close distance와 Loud speech 조건에서는 두 실험자 모두 거의 동일하게 10/10의 detection rate를 기록하였다.

이러한 결과는 깨끗한 환경에서는 화자 차이가 크게 중요하지 않지만, 신호 품질이 낮아질수록 화자의 발음, 음성 크기, 억양, 발화 속도 등의 특성이 detection 성능에 더 큰 영향을 미친다는 것을 의미한다. 실제로 배경 소음이 존재하는 환경에서는 작은 발음 차이도 wake word의 특징을 더 쉽게 가리게 되므로, 화자 간 성능 차이가 확대될 수 있다. 따라서 본 실험에서는 화자 간 variability가 존재한다고 볼 수 있으며, 특히 음악이나 주변 소음이 있는 어려운 환경에서 그 차이가 더욱 두드러지게 나타났다.

3. 현재 시스템은 약 50 cm 이내 거리에서 일반적인 음량 이상으로 발화할 경우 안정적으로 동작한다. 또한 조용한 실내 환경에서는 높은 detection rate를 유지할 수 있다. 반면 1 m 이상의 거리, 매우 작은 음량, 그리고 음악이나 실험실 소음과 같은 강한 배경 소음이 존재하는 환경에서는 성능이 크게 저하되었다. 따라서 현재 시스템을 사용할 때는 마이크와 비교적 가까운 거리에서 일반적인 음량 이상으로 발화하는 것이 권장된다. 실제 사용 환경으로는 조용한 실내 공간이나 개인 작업 공간이 적합하며, 시끄러운 공용 공간이나 음악이 재생되는 환경에서는 안정적인 wake word detection을 기대하기 어려울 것으로 판단된다.

[Task 4.1]

1. 본 실험에서는 false positive scenario로 "Hey testing!"을 선택하였다.

이 phrase를 선택한 이유는 이전 실험(Task 3.2)에서 사용한 "Hey question!"이나 "Hey Christian!"과 같은 strong adversarial examples는 거의 항상 wake word로 인식되었기 때문이다. 이러한 경우에는 parameter를 조정하더라도 false positive를 제거하기 어렵기 때문에 parameter optimization의 효과를 관찰하기 적합하지 않다.

반면 "Hey testing!"은 몇 차례 false positive 실험을 해본 결과 wake word와 일부 음운적 유사성("Hey", "ting")을 가지면서도 항상 detection되지 않는 weak adversarial example이었다. 따라서 threshold와 CONSECUTIVE_DETECTIONS를 변경했을 때 false positive와 true positive가 어떻게 변화하는지 관찰하기에 적절한 시나리오라고 판단하였다.

Threshold	True positives	False positives	Notes
0.5	20/20	9/20	Very sensitive; highest false positive rate
0.6 (default)	20/20	7/20	Default setting; still prone to false positives
0.7	20/20	8/20	Similar performance to 0.6; limited improvement
0.8	17/20	1/20	Best balance between true positives and false positives
0.9	8/20	0/20	Many wake words missed

2. Threshold가 증가할수록 전반적으로 false positive는 감소하는 경향을 보였다. 특히 threshold를 0.8로 설정하였을 때 false positive는 1회까지 감소하였으며, true positive 역시 17/20으로 비교적 높은 수준을 유지하였다.

반면 threshold를 0.9까지 증가시키면 false positive는 완전히 제거되었지만 true positive가 8/20까지 감소하였다. 이는 실제 wake word조차 상당수 놓치게 됨을 의미한다.

따라서 false positive 감소와 true positive 유지 사이의 균형을 고려할 때, 본 실험에서는 0.8을 최적 threshold로 선택하였다.

Consecutive frames	True positives	False positives	Notes
1	15/20	2/20	Fast response but unstable
2	18/20	4/20	High sensitivity; more false positives
3 (default)	17/20	1/20	Best overall balance
4	10/20	1/20	Detection becomes overly strict
5	9/20	0/20	Lowest false positives but poor recall

3. Threshold를 0.8로 고정한 상태에서 CONSECUTIVE_DETECTIONS 값을 변경하였다. Consecutive frame 수가 증가할수록 false positive는 감소하는 경향을 보였지만, 동시에 true positive도 감소하였다. 특히 4 이상에서는 실제 wake word detection 성능이 급격히 저하되었다.

Consecutive frame 수를 1 또는 2로 설정하면 true positive는 높은 편이었지만 false positive가 상대적으로 많이 발생하였다. 반면 5로 설정하면 false positive는 제거되었으나 true positive가 9/20에 불과하였다.

따라서 본 실험에서는 false positive와 true positive 사이의 균형이 가장 우수한 CONSECUTIVE_DETECTIONS = 3을 최적값으로 선택하였다. 또한 이 값에서는 detection latency 증가는 체감이 거의 되지 않았다.

4. Discussion

What is the expected performance?

추천 설정(Threshold = 0.8, Consecutive Detections = 3)에서는 true positive 17/20(85%)와 false positive 1/20(5%)의 결과를 얻었다. 따라서 대부분의 정상적인 wake word는 성공적으로 검출하면

서도 불필요한 활성화는 크게 줄일 수 있을 것으로 예상된다.

What types of user scenarios or use contexts would this level of performance support?

현재 시스템은 조용한 실내 환경이나 개인 작업 공간과 같이 비교적 낮은 배경 소음 환경에서 사용하기 적합하다. 또한 사용자가 마이크 근처에서 일반적인 음량 이상으로 발화하는 경우 안정적으로 동작할 것으로 예상된다.

반면 Task 3.3의 결과에서 확인한 것처럼 먼 거리(1 m 이상), 매우 작은 음량, 그리고 강한 배경 소음 환경에서는 성능 저하가 발생할 수 있다.

What are the key error scenarios? When would you expect errors to occur?

주요 오류 상황은 작은 음량, 먼 거리, 그리고 강한 배경 소음 환경이다. 실제로 Quiet speech 조건에서는 두 실험자 모두 1/10의 detection rate를 기록하였으며, Lab noise 환경에서도 성능 저하가 관찰되었다.

또한 wake word와 발음이 유사한 phrase가 포함된 대화에서도 false positive가 발생할 가능성이 있다. 특히 음운적 구조가 유사한 adversarial phrase는 parameter tuning 이후에도 일부 오동작을 유발할 수 있다.

How could performance be improved?

추가적인 threshold 및 consecutive frame tuning을 통해 특정 환경에 맞는 성능 최적화가 가능하다. 그러나 parameter tuning만으로는 strong adversarial examples에 의한 false positive를 완전히 제거하기 어렵다.

보다 근본적인 성능 향상을 위해서는 다양한 환경과 화자를 포함하는 데이터셋을 활용하여 모델 자체를 개선하거나, 보다 정교한 wake word detection 모델을 사용하는 방법이 필요할 것으로 판단된다.

How does the performance of the system compare to commercial examples?

상용 voice assistant는 장시간 대기 상태에서도 매우 낮은 false positive rate와 높은 wake word detection 성능을 제공한다. 반면 본 실험 시스템은 특정 phrase와 배경 소음 환경에서 여전히 오동작이 발생하였다. 따라서 본 시스템은 실험용 wake word detector로는 충분히 의미 있는 성능을 보였지만, 상용 제품 수준의 안정성과 신뢰성에는 아직 미치지 못한다고 판단된다.